

Progress Report: Zero-Shot Violation Detection on Real Client Data

Project: AI Adoption Clinic — Municipal Code Enforcement for Riverdale Park **Student:** Chihao Li (Technical Lead) **Date:** 2026-04-13 **Scope:** Phase 1 + Phase 2 consolidated report covering model evaluation on the first batch of real inspector photos

Background

Our team is building an image classification system for the Town of Riverdale Park’s Development Services department. The client, Director Ryan Chelton, sees the same pattern every week: an inspector photographs a property violation in the field, then spends meaningful time back at Town Hall mapping the image to the correct municipal code section. He wants AI to take over the mapping step — accept a photo and return the matching code (e.g., § 304.7 Damaged Roof Shingles), leaving the inspector to confirm or reject.

Client data has been the central constraint of the project all year. Compliance review at Town Hall has blocked release of most of the inspection archive, and when the first real batch finally arrived, it was small: **98 photos across 14 official violation codes**, of which only 9 classes actually contained images. Photos are organized by folder, one folder per code.

This data volume is far below what supervised training would need. The project has therefore committed to a **zero-shot approach** — use off-the-shelf foundation models and build everything around their pretrained knowledge, without fine-tuning on client data.

Prior Work (Phase 1, Week of 2026-04-06)

Phase 1 evaluated two zero-shot models on the 98-image client set: **CLIP ViT-B/32** (LAION-2B) as an embedding-based baseline, and **Gemma 4 E4B-IT** (4-bit quantized via MLX), Google’s April-2026 open-weight multimodal VLM, run locally on an M4 24 GB Mac.

Both models used a multi-class classification framing: for each image, produce a ranked list of the 9 candidate violation codes. Results under a multi-label ground truth (explained below):

Model	Top-1	Top-3
CLIP ViT-B/32	80.6%	94.9%
Gemma 4 E4B-IT	85.7%	96.9%

The numbers were respectable, but Phase 1’s most important finding was not numerical. While debugging individual error cases, I discovered that **11 photos in the client data were byte-identical copies filed in multiple category folders simultaneously** (verified by MD5 hash). One photo of a burned-out building appeared under Graffiti, Long Grass, AND Damaged Roof

Shingles — the inspector had tagged it with three codes. This established that the client’s dataset is **fundamentally multi-label**, even though it is organized as single-label folders.

A second finding came from Gemma’s per-class errors. On boarded_windows, Gemma’s recall collapsed to 25% — but its free-text rationale clearly described the correct visual features (“weathered wood, peeling paint, broken shingles”). The model was seeing everything correctly and then selecting a more specific class in its top-1 position because of over-specificity pressure inside the argmax. **The failure was at the reasoning layer, not the perception layer.**

Combining these two observations, Phase 1 concluded that the task itself was framed wrong. The inspector’s actual decision is not “which class does this photo belong to?” but “for each code, can I cite this photo?” The first framing is multi-class classification; the second is **per-class binary detection**. Despite looking mathematically similar, they have completely different evaluation metrics, different prompt structures, and different failure modes.

Phase 2 Objective

Phase 1 left three hypotheses that were plausible on reasoning but unvalidated empirically:

1. **Would per-class binary detection actually work better than multi-class classification?** Same models, same data — only the prompt format changes. Would Gemma’s boarded_windows collapse recover?
2. **Could CLIP and Gemma be combined into a cascade — CLIP as candidate generator, Gemma as per-candidate verifier — that outperforms either model alone?** Phase 1 showed the two models had complementary per-class strengths, but the cascade was only a design sketch.
3. **Is the lingering precision problem on peeling_paint and overgrown_vegetation a model failure, or is it ground truth incompleteness?** If inspectors tag only the primary violation and ignore background violations, our “precision” metric is systematically penalizing models for seeing things the ground truth omits.

Phase 2 was designed to convert all three from reasoning to measurement.

Method

Phase 2 conducted three connected experiments.

Experiment 1: CLIP per-class separability. Rather than passing the CLIP output through softmax and argmax, I preserved the raw 98×9 cosine similarity matrix. Each column became an independent “CLIP’s opinion on a single class.” From this matrix I computed **top-k hit-any recall** — for each image, does CLIP’s top-k candidate list contain any of the image’s valid labels? This is the correct metric for a cascade’s Stage 1, because Stage 1’s job is to route candidates to Stage 2, not to make final decisions.

Experiment 2: Gemma 4 per-class binary verification. I wrote a new prompt that asks one question at a time: “Does this image show <class description>?” with an enforced single-line JSON response format containing answer (yes/no), confidence (0-100), and rationale (short

sentence). For each of the 98 images, I ran Gemma on all 9 classes independently — **882 total calls** at roughly 3 seconds each. The inference script streams each result to a JSONL file and supports resume-from-crash: if interrupted, rerunning the script automatically skips already-completed (image, class) pairs. This proved essential — the overnight run encountered MPS thermal throttling after the laptop went to sleep, and I interrupted-and-resumed it once without data loss.

Experiment 3: CLIP × Gemma cascade. For each image, take CLIP’s top-k candidates, run Gemma’s binary query on each one, threshold at confidence ≥ 0.5 , and emit the union as the final prediction set. Evaluate with Jaccard similarity and sample-averaged F1 against the multi-label ground truth. Compare cascade F1 at k=3, 5, and 9 (k=9 reducing to a standalone Gemma binary baseline).

All three experiments share the same 98-image client set and the same MD5-derived multi-label ground truth. No training, no fine-tuning, no external data.

Results

CLIP as a Stage 1 candidate generator performs well in aggregate:

k	Hit-any recall	Missed images
1	80.6%	19
3	94.9%	5
5	99.0%	1
6	100.0%	0

At k=5, CLIP captures the correct answer for 97 of 98 images, making it an effective first-stage filter. However, per-class inspection revealed one systematic failure: `damaged_roof_shingles` has only 40% top-3 and top-5 recall. CLIP never ranks this class in its top 5 for 6 of 10 true positives. Other classes reach 100% recall by top-5. This is a concrete, reproducible Stage 1 bottleneck.

Gemma 4 per-class binary verification produced a fundamentally different error profile than the Phase 1 ranked version. All 882 calls returned valid JSON (100% parse rate), and per-class results at threshold 0.5:

Class	AUC	Recall	Precision
deteriorating_chimney	1.000	1.00	0.75
graffiti	0.993	1.00	0.67
inoperable_vehicle	0.977	0.90	0.72
junk_trash_accumulation	0.922	1.00	0.36
overgrown_vegetation	0.904	1.00	0.48
damaged_roof_shingles	0.850	0.70	0.50
peeling_paint	0.796	1.00	0.27
broken_windows	0.787	0.54	0.64
boarded_windows	0.764	0.60	0.43

Two patterns stand out. First, **recall is broadly high** — 5 of 9 classes reach 100% recall, and boarded_windows recovered from Phase 1’s 25% to 60% simply by changing the prompt from ranked to binary. This directly confirms Phase 1’s hypothesis that the failure was reasoning, not perception. Second, **precision is low on four classes** (peeling_paint 27%, junk_trash 36%, boarded 43%, overgrown 48%). Whether these are genuine errors or ground truth gaps is the central question of the interpretation below.

Per-class AUC comparison between CLIP and Gemma reveals strong complementarity:

Class	CLIP AUC	Gemma AUC	Stronger
overgrown_vegetation	0.709	0.904	Gemma (+19.5)
damaged_roof_shingles	0.775	0.850	Gemma (+7.5)
broken_windows	0.953	0.787	CLIP (+16.6)
boarded_windows	0.931	0.764	CLIP (+16.7)
peeling_paint	0.903	0.796	CLIP (+10.7)
chimney, graffiti, vehicle, junk_trash	~same	~same	tied

Gemma rescues CLIP on exactly the two classes CLIP struggled with in Phase 1. CLIP rescues Gemma on the three facade/window classes where Gemma’s over-specificity hurts. The two models’ error distributions are almost entirely disjoint, as shown in the side-by-side AUC comparison below:

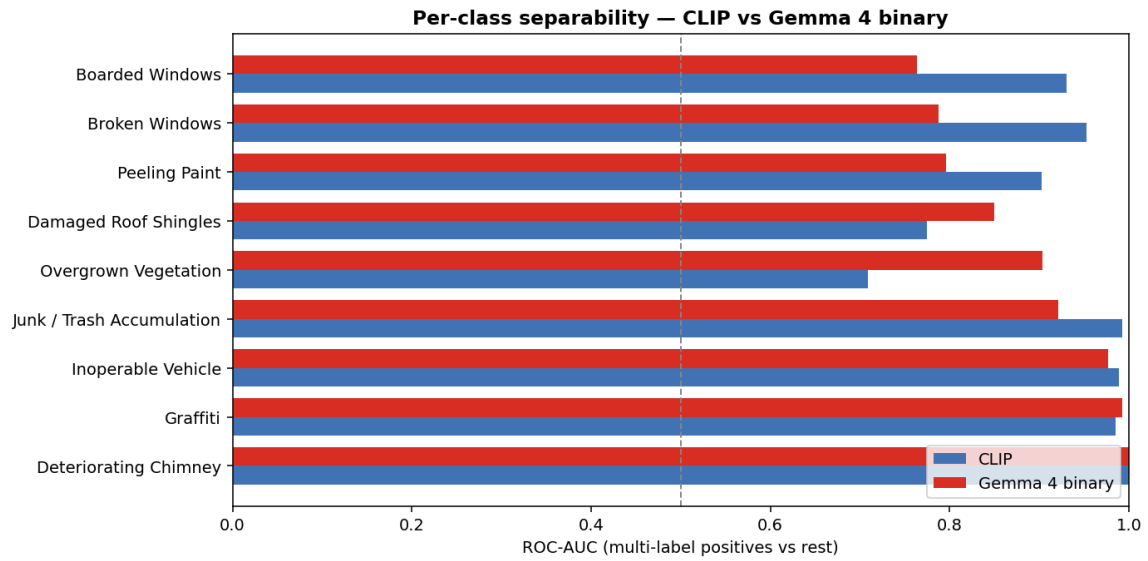


Figure 1: Per-class AUC: CLIP vs Gemma 4 binary

The underlying CLIP per-class score distributions (orange = positives, gray = leave-one-out negatives from other violation classes) make the separability pattern visible:

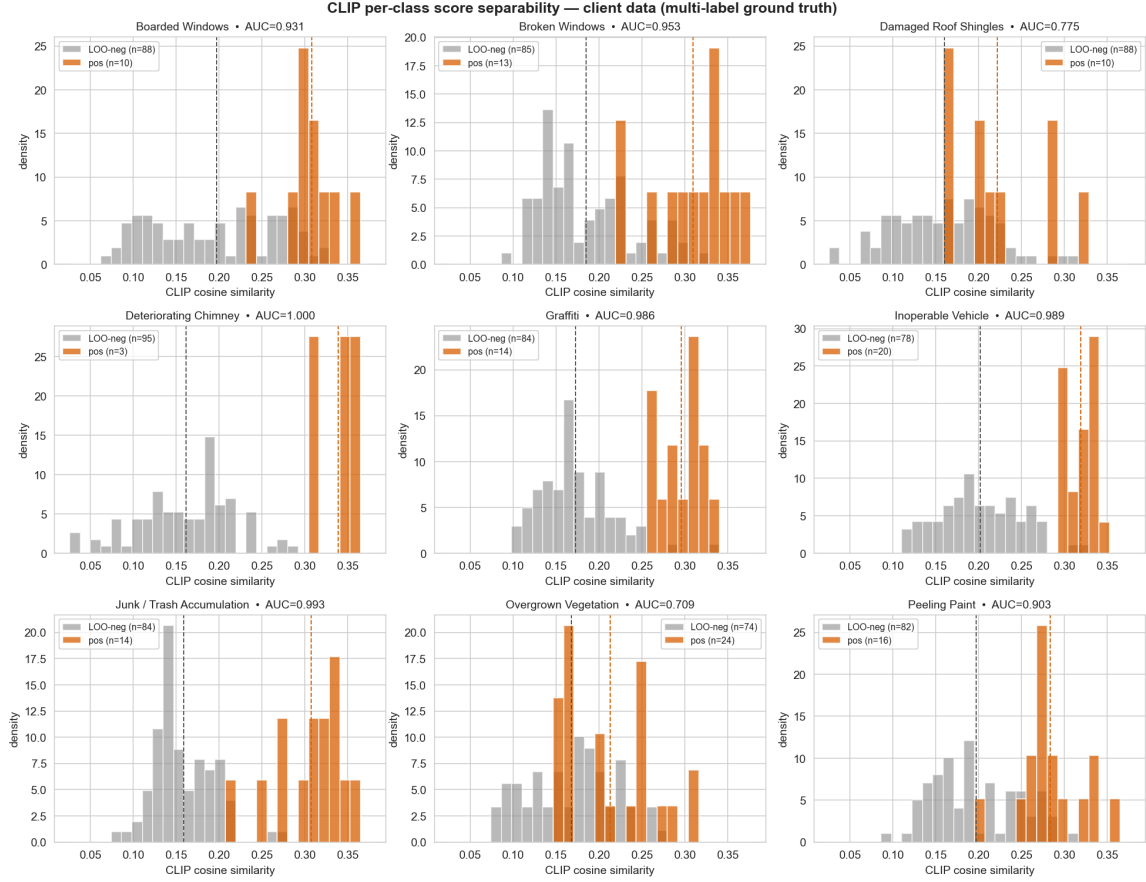


Figure 2: CLIP per-class score separability

The **cascade** outperforms either model alone:

Configuration	Jaccard	Sample-F1	Predictions/image
Gemma binary alone (= k=9)	0.525	0.633	2.43
Cascade k=5	0.568	0.668	2.08
Cascade k=3	0.622	0.703	1.60

A counter-intuitive finding: smaller k produces higher F1. k=3’s F1 of 0.703 beats k=5 (0.668) and k=9 (0.633). The expected pattern should have been “more candidates = more recall = more F1”, but the data shows the opposite.

The explanation is that CLIP’s “silence” — assigning low similarity to a class — functions as a veto on Gemma’s high-FPR classes (peeling_paint and overgrown_vegetation, where Gemma says “yes” to roughly half of the non-target images). At k=3, CLIP filters out most questions on which Gemma would have produced false positives, so Gemma never gets the chance to misfire. At k=9, every question reaches Gemma, and the high-FPR classes pollute the output. The cascade is

effectively a **joint verification system**: a flag requires agreement from two independent evidence sources, and their errors rarely coincide.

Conclusions

The framing reframe is validated. Gemma’s boarded_windows recall moved from 25% (Phase 1 ranked) to 60% (Phase 2 binary) with no model changes, no training, and no data changes — only the prompt format. Three other classes that were compressed by argmax in Phase 1 reached 100% recall in binary mode. The same observation holds for CLIP: per-class AUC on multiple classes is higher than the classes’ top-1 recall, meaning CLIP carries more information than the classification pipeline exposes. Task framing was the bottleneck, not model capacity.

The cascade is a joint verification mechanism, not an efficiency optimization. My initial mental model of the cascade was “CLIP as cheap filter, Gemma as expensive verifier, smaller k saves compute.” The data contradicted this: smaller k raises precision because it uses CLIP’s silence as a veto power over Gemma’s failure modes. The correct framing is “two independent foundation models must both agree before a flag is raised.” This reframing has implications for how we present the system to Ryan: not a hierarchical filter, but a multi-source consensus mechanism — a narrative closer to responsible-AI best practices than to standard engineering optimization.

The remaining precision gap is likely ground truth incompleteness. peeling_paint in cascade k=3 has 100% recall but only 40% precision, meaning most of Gemma’s “yes” votes are against photos that the client did not label as peeling paint. However, Riverdale’s inspector photos are overwhelmingly of deteriorating buildings, where peeling paint is a near-universal background condition. The client’s filing convention is “which code did the inspector cite?”, not “which codes are visually present?” Gemma is most likely identifying real but un-flagged background violations, which our ground truth reports as false positives. Resolving this requires either re-annotation of the existing 98 photos with full multi-label truth, or a set of truly negative (non-violation) reference photos — both of which are client-dependent data asks.

Implications

Task framing has higher ROI than hyperparameter tuning in this regime. Over Phase 1 and Phase 2 combined, every performance improvement came from changing how we asked the model — never from training, fine-tuning, a larger model, or hyperparameter changes. This is an important direction for the AI Adoption Clinic curriculum: when a foundation model underperforms, the first instinct should be to question the task definition, not to search for better weights.

“Two independent questions” beats “one compound question” for foundation models. The ranked prompt forced Gemma into internal competition among 9 classes, implicitly producing a ranking, which the model did poorly because of over-specificity pressure. Splitting into 9 independent yes/no questions removed the implicit ranking step and let the model judge each class on its own. This is a general pattern — whenever a task can be decomposed into independent sub-questions, decomposition should be preferred over letting the model do internal comparison.

Error complementarity is the right basis for ensemble design. CLIP and Gemma make almost entirely non-overlapping errors on this dataset. This was not by architectural design (one is contrastive, the other generative — these categories predict nothing about error correlation); it was an empirical discovery. Future ensemble design decisions should be driven by measured error correlation rather than by architectural intuition about “which models should complement each other.”

Next Steps

Immediate work (no client dependency):

1. Rewrite the CLIP prompt for `damaged_roof_shingles`. The current three prompts are descriptive; replacing them with more visually specific phrasing may lift the class into top-5 and eliminate the one Stage 1 choke point.
2. Tighten the `peeling_paint` binary prompt with qualifiers like “substantial” or “noticeable area” to test whether FPR drops without hurting recall. If the drop is flat, it strongly supports the ground-truth-incompleteness interpretation.
3. Consolidate the cascade into a single `CascadePipeline` module that the notebook and a CLI demo can call, to support live demonstration at the next client meeting.

Client data asks (to be raised at the next Ryan Chelton sync):

1. Approximately 200 “no violation” photos — buildings an inspector visited and judged compliant. These are the only way to compute operationally meaningful precision and false positive rate.
2. Full multi-label re-annotation of the existing 98 photos, with inspectors marking every visible violation rather than only the primary cited one. This would empirically resolve the `peeling_paint` precision paradox.

The previous data ask was the generic “give us more labeled data.” The new ask is specific: negatives for FPR measurement, and completion of existing labels for precision measurement. This framing is more likely to pass the compliance review that has been blocking the original full archive.

Not recommended:

- Training DINOv2 or other supervised models. With a cascade F1 of 0.703 achieved at zero training cost, the ROI on training is low until meaningfully more client data is available.
 - Expanding the taxonomy. The 9-class pipeline is already sufficient to demonstrate extensibility (adding a new code = writing one prompt), and which specific codes to add should be driven by Ryan’s business priorities rather than technical path-of-least-resistance.
-

Reproducibility

All code, scripts, and report sources are public at <https://github.com/psylch/inst623-riverdale-code-enforcement>. To reproduce the experiments described above from a clean machine:

```

# Clone the repository and enter the project root
git clone https://github.com/psylch/inst623-riverdale-code-enforcement.git
cd inst623-riverdale-code-enforcement

# Install Python dependencies (uses uv for Python 3.12)
uv sync

# Place client photos under data/client-data/ before running
# (one subdirectory per violation code, matching the folder names in the
# official Riverdale Park taxonomy). The client dataset itself is not
# redistributed in this repository for privacy reasons.

# CLIP separability + top-k analysis (~2 minutes)
uv run python scripts/run_clip_separability.py

# Gemma binary verification (~47 minutes, resumable)
uv run python scripts/run_gemma_binary.py
# Monitor progress in another terminal:
tail -f checkpoints/client_gemma4_binary_stream.jsonl

# Cascade evaluation and figures
uv run python scripts/analyze_binary_results.py

```

Every path above is relative to the repository root. All experimental artifacts (predictions, similarity matrices, audit logs, and figures) land under checkpoints/ and reports/figures/. The full pipeline runs end-to-end on an Apple Silicon Mac with 24 GB unified memory — model weights are downloaded from Hugging Face on first run and cached locally, and no external compute is required.